We confirm and declare that this report and the project work is entirely the product of our own efforts and we have not used or presented the work of others herein without acknowledgement

EE580 Mini Project

01 April 2026

Department of Electronic and Electrical Engineering
University of Strathclyde, Glasgow

Daniel Stebbings
reg. 202118874

Preben Rasamanickam
reg. 202114642

Contents

1 Introduction **1**

2 Methodology **1**
2.1 MATLAB Filter Design and Validation 1
2.2 Embedded System Implementation 1
2.2.1 State Machine 1
2.2.2 Buffer and Memory Management 2
2.2.3 Filter Integration 2
2.3 Optimisation and Profiling 2

3 Results **3**
3.1 Embedded System Filter Results 3
3.2 Embedded System Profiling Results 3

4 Discussion **3**

5 Conclusion **4**

6 Figures and Tables **5**
6.1 Embedded Systems 5
6.1.1 Audio Signals Results 5
6.1.2 Profiling Results 9

7 Code Listings **10**
7.1 MATLAB Implementation 10
7.2 C Implementation 10
7.2.1 cbuf.h 10
7.2.2 cbuf.c 11
7.2.3 iir.h 14
7.2.4 iir.c 16
7.2.5 state_machine.h 18
7.2.6 state_machine.c 19
7.2.7 framework.h 27
7.2.8 framework.c 28
7.2.9 profiling.h 37
7.2.10 profiling.c 37
7.2.11 main.c 37

8 Bibliography **41**

9 Contributions **41**

1 Introduction

This project involved the design and implementation of a real-time embedded Digital Signal Processing (DSP) system using the TMS320C6748 DSP and OMAP-L138 Experimenter board. The primary objective was to acquire, store, and process audio signals through a user-controlled interface in real-time.

The system architecture can be divided into three functional states based on the DIP switch configuration:

- Standby: The system remains idle.
- Record: The system captures the latest 4 seconds of audio from the LINE IN into a circular buffer while providing a loopback to the LINE OUT.
- Playback: The system plays back the recorded 4-second buffer in a circular fashion.

During playback, the user can apply three distinct Infinite Impulse Response (IIR) filters to the buffered audio - A Low-Pass (LP), Band-Pass (BP), and High-Pass (HP), applied individually or in combination. The filters were designed to meet the following frequency specifications based on a sampling frequency of f_s = 8 kHz:

- LP: Cut-off at $f_1 = \frac{f_s}{6}$
- BP: Pass-band between f_1 and $f_2 = \frac{f_s}{3}$
- HP: Cut-off at $f_2 = \frac{f_s}{2}$

Multiple optimisation methods were investigated, including Single Instruction Multiple Data (SIMD) instructions, pragmas, and loop unrolling, and profiling was conducted to evaluate the cycle-count efficiency of the optimised implementation compared to the baseline version.

2 Methodology

2.1 MATLAB Filter Design and Validation

The initial phase involved designing three stable Infinite Impulse Response (IIR) filters using MATLAB Filter Designer to meet the previously specific spectral requirements.

2.2 Embedded System Implementation

The system was developed in Code Composer Studio (CCS) for the TMS320C6748 DSP amd is implemented on the OMAP-L138 Experimenter board. The software architecture uses DSP/BIOS to manage real-time scheduling and hardware/software interrupts. The implementation is split between audio processing and lower-priority state management to ensure the 8kHz sampling rate is maintained for real-time performance.

2.2.1 State Machine

The system's behavior is controlled by a state machine that transitions between three primary states based on the status of DIP switches S2:1 and S2:2. This is implemented via two main functional blocks:

- **State Transition Logic** (dipState): A periodic function executing every 50ms to poll the DIP switches and update the global state variable g_sys_state. The system can have the following states:

    - **Standby**: Triggered when S2:1 is off. The system ignores other switches, turns off LEDs, and ensures no audio is output.
    - **Record**: Triggered when S2:1 is on and S2:2 is off. The system initialises the circular buffer, and records and outputs live audio.
    - **Playback**: Triggered when both S2:1 and S2:2 are on. If the circular buffer is full, the system transitions to processing the stored audio based on the filter selection. After processing, the system outputs the processed audio data in a circular fashion.

- **Real-Time Audio Handling** (audioHWI): This Hardware Interrupt Service Routine (ISR) executes at the sampling frequency. It handles the audio input/output of the system, performing the following based on g_sys_state:
    - **Standby**: A zero-valued sample is written to the codec.
    - **Record**: The system enqueues the incoming sample into the circular buffer and simultaneously routes it to the LINE OUT for loopback.
    - **Playback**: The next sample is fetched from the processed audio output buffer (g_audio_out) and writes it to the codec.

2.2.2 Buffer and Memory Management

To store 4 seconds of audio at 8kHz, a circular buffer structure was developed to manage 32000 16-bit samples.

- Circular Buffering: During the Record state, the buffer always containing the most recent 4 seconds of audio.
- When the buffer is full, newest sample is added at the tail position, the tail and head position are advanced simultaneously, which overwrites the oldest sample.
- This implementation avoids memory shifts which have a high hardware cost.

To optimize memory usage, the processed audio output is produced by only processing smaller chunks of the circular buffer at a time. As a result, much smaller intermediary processing buffers can be used. This adds a small processing delay, which can be addressed by optimisation measures such as SIMD intrinsics and loop unrolling.

2.2.3 Filter Integration

During Playback, the system polls DIP switches S2:6-8 to identify which filters (LP, BP, HP) are active.

- Parallel Summation: If multiple filters are selected, the input chunk is passed through each active filter independently. The resulting outputs are summed together before being stored for playback.

2.3 Optimisation and Profiling

To achieve maximum real-time performance, the baseline IIR implementation was refined into an optimised version. Several hardware-specific techniques were employed:

- Compiler Pragmas: #pragma UNROLL and #pragma MUST_ITERATE provided the compiler with loop count guarantees for software pipelining and loop unrolling.
- SIMD and Intrinsics: The following intrinsics were used for more efficient data operations:
    - amem4: Accessed 32-bit words (two 16-bit samples) simultaneously.

- _spint and _spack2: Efficiently converted floating-point results back to integer format with saturation to prevent overflow.
- _sadd2: Performed addition on two packed 16-bit integers in a single cycle.

The performance impact of these optimisations was measured using the clock() function to capture the exact cycle counts for all seven filter permutations.

3 Results

3.1 Embedded System Filter Results

The frequency domain plot of the audio input buffer contents can be seen in Figure 1. The input signal has a broad spectral distribution of the sampled audio with a larger weighting of low-frequency components.

The result of applying the different permutations of filter configurations can be seen in the following figures:

- LP Figure 2
- BP: Figure 3
- HP: Figure 4
- LP+BP: Figure 5
- LP+HP: Figure 6
- BP+HP: Figure 7
- LP+BP+HP: Figure 8

3.2 Embedded System Profiling Results

The profiling results of the IIR filter implementations are summarised in Table 1. The data compares the total CPU cycles required to process the 4-second audio buffer using the baseline filter application function versus the optimised version.

As shown in the table, the optimised implementation consistently improves the CPU runtime by approximately 34% across all active filter configurations. This performance gain is achieved primarily through loop unrolling and usage of SIMD intrinsics.

4 Discussion

The experimental data confirms that the OMAP-L138 Experimenter board is sufficiently capable of real-time audio processing when system features like hardware/software interrupts are leveraged.

The 34% improvement in cycle counts highlights the effectiveness of several key techniques:

- Loop Unrolling and Pragmas: By using #pragma UNROLL and #pragma MUST_ITERATE, the compiler was able to schedule multiple instructions in parallel, taking advantage of the multiple execution units.
- SIMD Intrinsics: The use of _amem4 allowed for simultaneous loading of two 16-bit samples into a single 32-bit register, effectively doubling the memory throughput. While _sadd2 and _spack2 allowed for addition of four 16-bit samples in a single instruction, while providing hardware-level saturation.

5 Conclusion

This project successfully demonstrated the design and implementation of a real-time embedded digital signal processing system using the TMS320C6748 DSP and OMAP-L138 Experimenter board. By integrating MATLAB-designed IIR filters with an optimised C implementation in CCS, the system effectively managed audio acquisition, circular buffering, and frequency-selective playback at a sampling rate of 8 kHz.

Key findings and achievements from the project include:

- The implementation of a state machine allowed for seamless transitions between Standby, Record, and Playback modes via DIP switch user control.
- Use of compiler pragmas, circular buffers, and SIMD intrinsics significantly enhanced the processing efficiency.
- Profiling execution confirmed that hardware-specific optimisations reduced CPU cycle consumption by approximately 34% compared to the baseline implementation, ensuring the system comfortably met real-time processing constraints.

6 Figures and Tables

6.1 Embedded Systems

6.1.1 Audio Signals Results

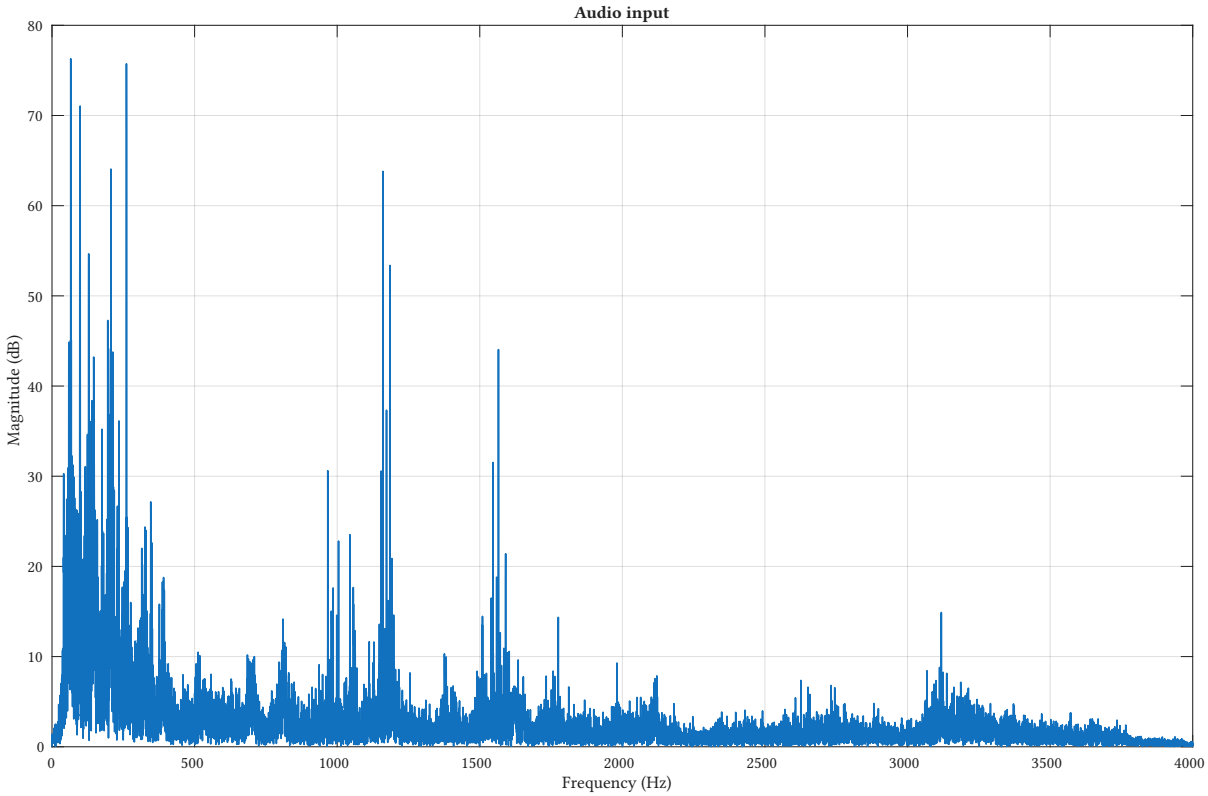

Figure 1: Input audio signal in frequency domain.

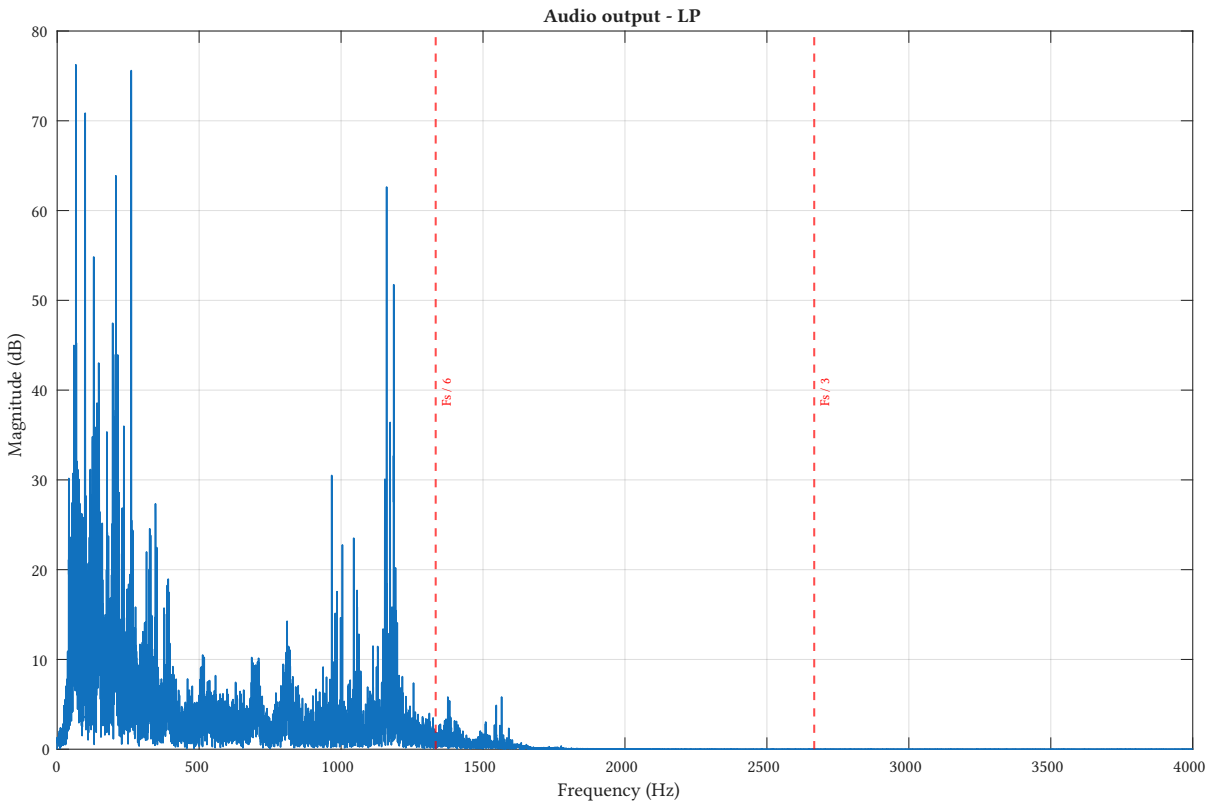

Figure 2: Frequency domain plot of audio filtered with the LP filter.

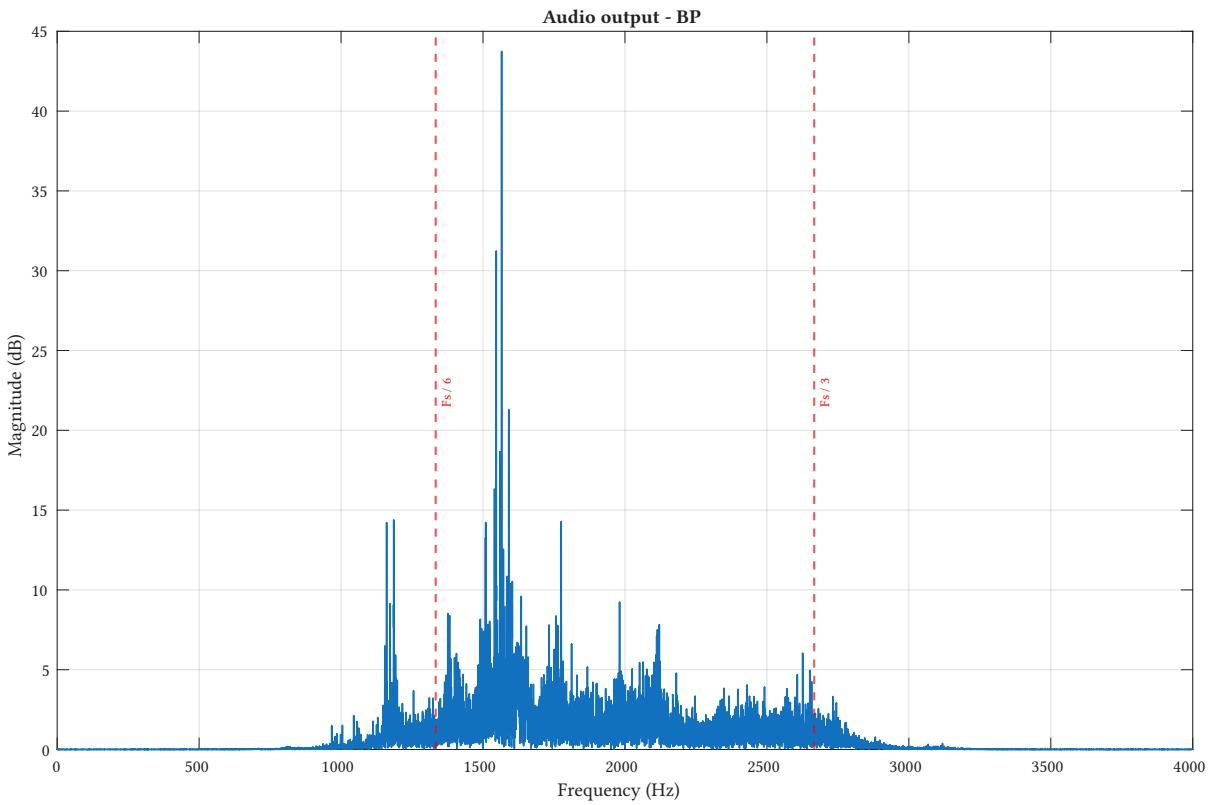

Figure 3: Frequency domain plot of audio filtered with the BP filter.

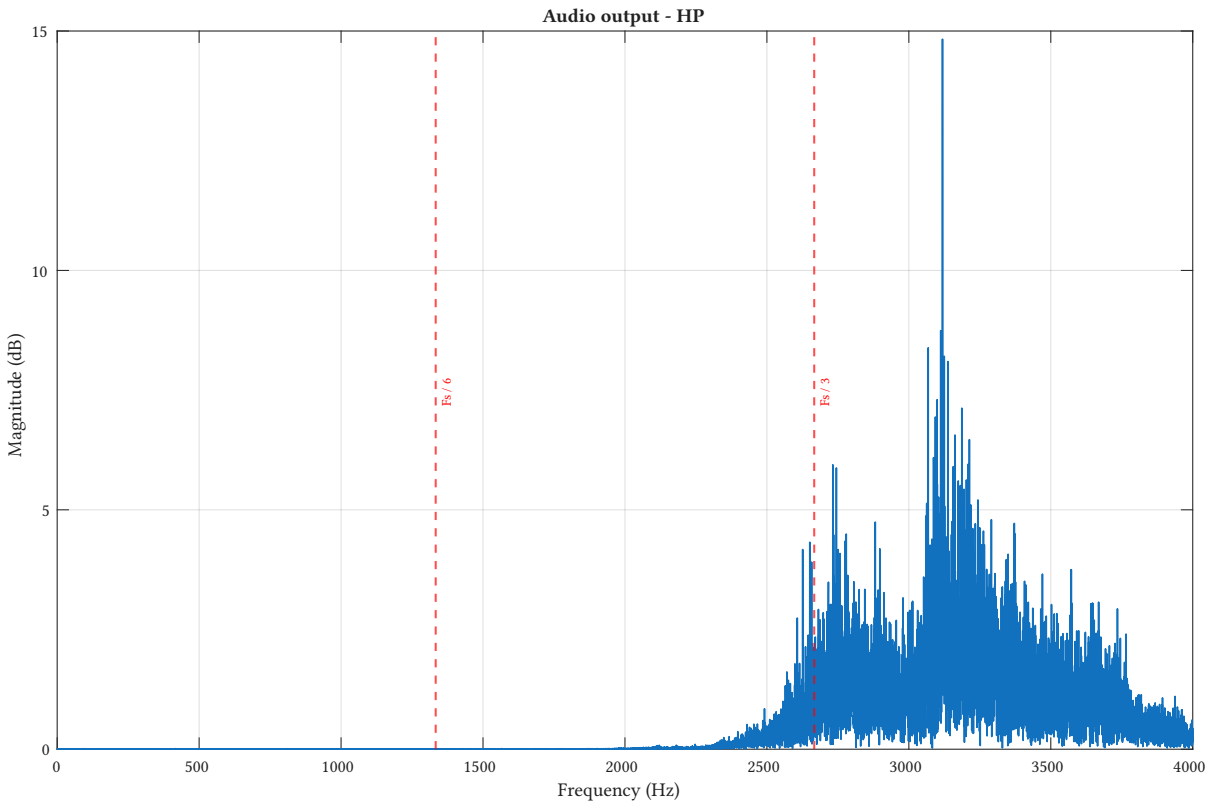

Figure 4: Frequency domain plot of audio filtered with the HP filter.

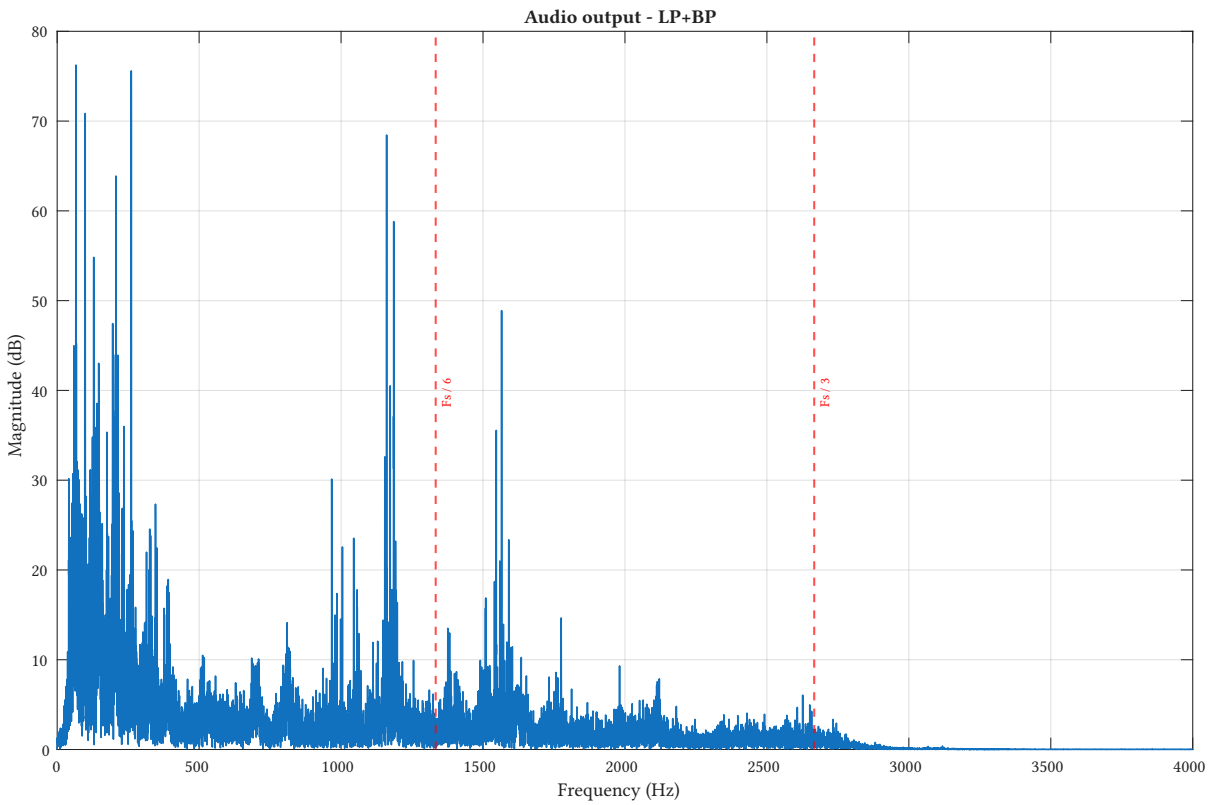

Figure 5: Frequency domain plot of audio filtered with combined LP and BP filters.

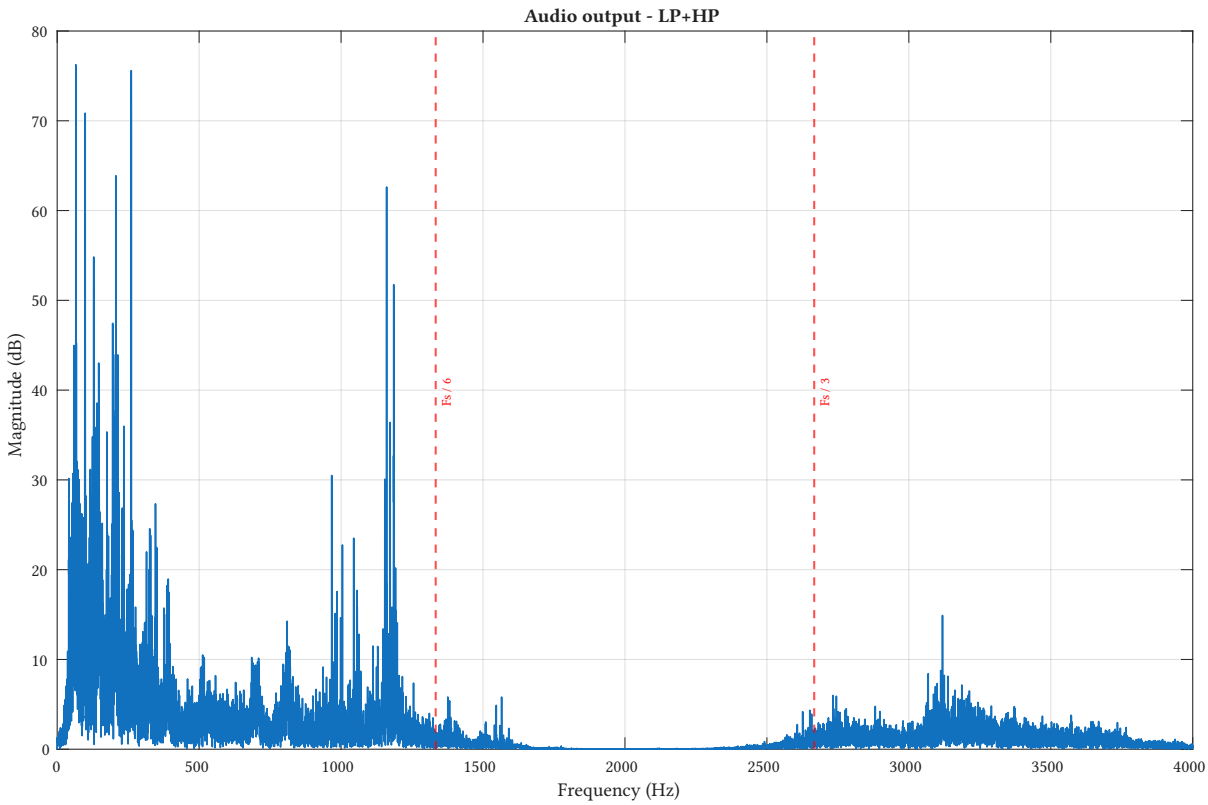

Figure 6: Frequency domain plot of audio filtered with combined LP and HP filters.

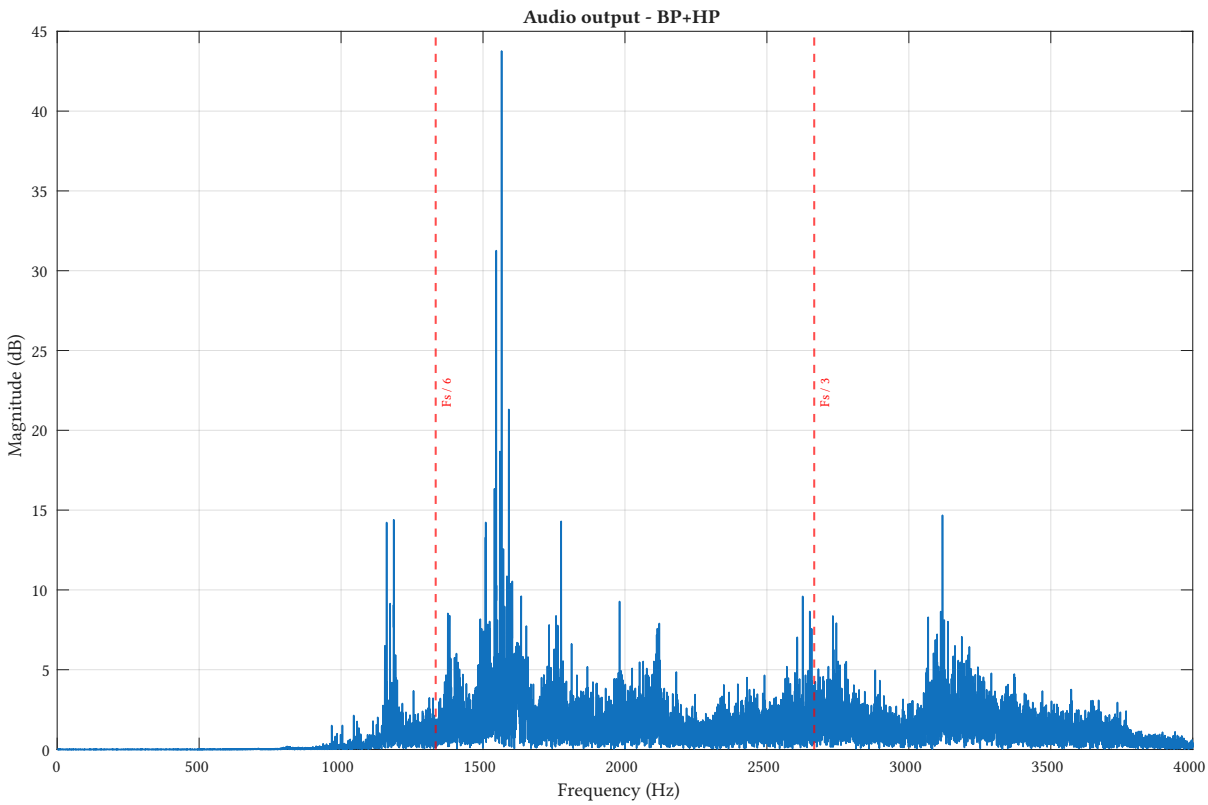

Figure 7: Frequency domain plot of audio filtered with combined BP and HP filters.

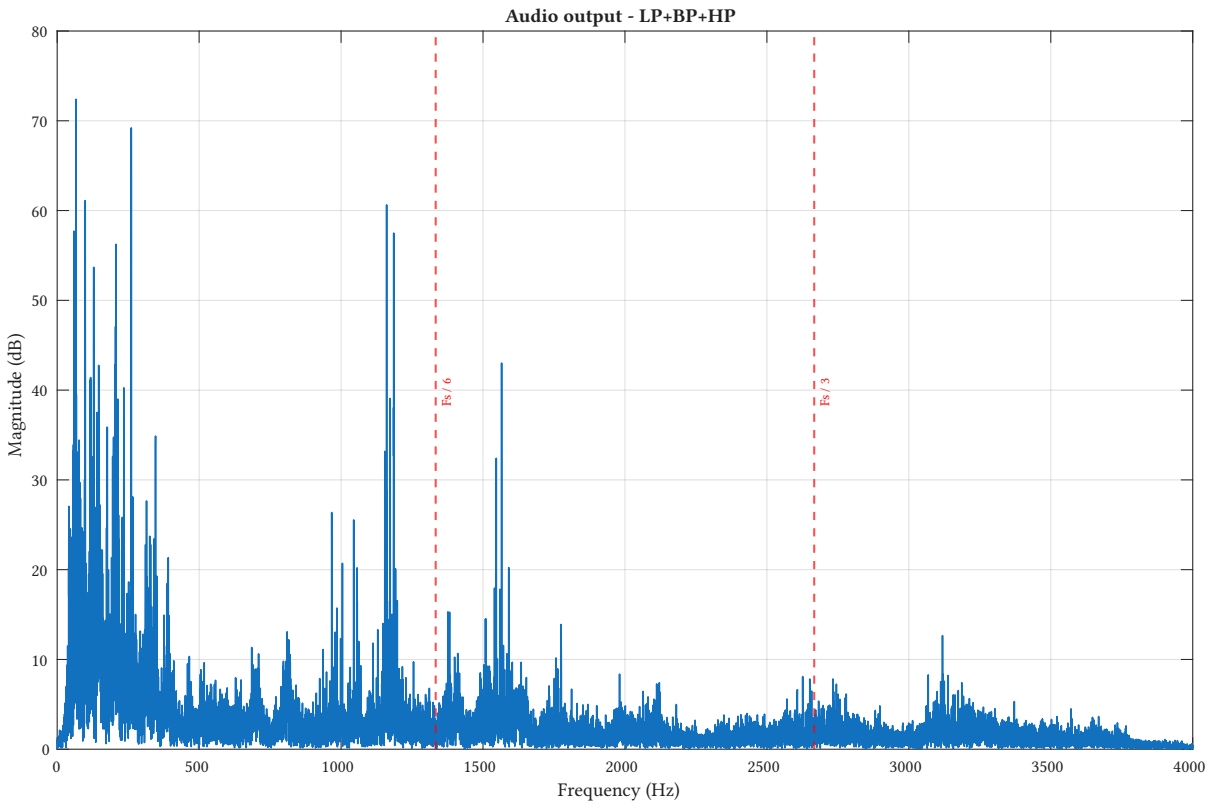

Figure 8: Frequency domain plot of audio filtered with all three filters (LP+BP+HP) applied.

6.1.2 Profiling Results

Table 1: Performance comparison of IIR filter implementations

HP	BP	LP	**Non-Opt Cycles**	**Opt Cycles**	**Ratio**
0	0	0	148527	146389	0.9856
0	0	1	53206075	35400123	0.6653
0	1	0	53165414	35431354	0.6664
1	0	0	53173689	35407980	0.6659
0	1	1	102647076	67620590	0.6588
1	0	1	102649226	67595442	0.6585
1	1	0	102654087	67627048	0.6588
1	1	1	152126303	99803334	0.6561

7 Code Listings

7.1 MATLAB Implementation

7.2 C Implementation

7.2.1 cbuf.h

```c
#ifndef CBUF_H
#define CBUF_H

#include <stdint.h>
#include <stdbool.h>

#define SAMPLE_RATE 8000       // hertz
#define RECORDING_TIME_S 4     // seconds
#define BUFFER_SIZE SAMPLE_RATE * RECORDING_TIME_S

typedef struct
{
    int16_t*        buffer;
    int32_t         head;
    int32_t         tail;
    int32_t         count;
    const int32_t   size;
} cbuf_t;

// Initialize the buffer
void init_buffer(cbuf_t *p_cbuf, int16_t* p_buffer);

// Check if the buffer is full
bool is_full(cbuf_t *p_cbuf);

// Check if the buffer is empty
bool is_empty(cbuf_t *p_cbuf);

// Add an element to the buffer
bool enqueue(cbuf_t *p_cbuf, int16_t p_sample);

// Remove an element from the buffer
bool dequeue(cbuf_t *p_cbuf, int16_t *p_sample);

int32_t peek_at(cbuf_t *p_cbuf, int16_t *p_bufout, int32_t p_n_samples,
int32_t p_start_idx);
int32_t peek_at_float(cbuf_t *p_cbuf, float *p_bufout, int32_t p_n_samples,
int32_t p_start_idx);

#endif // CBUF_H
```

7.2.2 cbuf.c

```c
/**
 * @file cbuf.c
 * @brief Circular buffer implementation
 */
#include "cbuf.h"
#include <stdbool.h>


// Initialize buffer
void init_buffer(cbuf_t *p_cbuf, int16_t* p_buffer)
{
    p_cbuf->buffer = p_buffer;
    p_cbuf->count = 0;
    p_cbuf->head = 0;
    p_cbuf->tail = 0;
    // p_cbuf->size set when initializing cbuf_t
}

bool is_full(cbuf_t *p_cbuf)
{
    return p_cbuf->count == p_cbuf->size;
}

bool is_empty(cbuf_t *p_cbuf)
{
    return p_cbuf->count == 0;
}

bool enqueue(cbuf_t *p_cbuf, int16_t p_sample)
{
    // null ptr return
    if (!p_cbuf) return false;

    // add sample at buffer tail
    p_cbuf->buffer[p_cbuf->tail] = p_sample;
    // advance tail with wrap-around
    p_cbuf->tail = (p_cbuf->tail + 1) % p_cbuf->size;

    if (p_cbuf->count == p_cbuf->size)
    {
        // buffer is full, oldest sample has been overwritten
        p_cbuf->head = (p_cbuf->head + 1) % p_cbuf->size;

    } else {
        // increment count.
        p_cbuf->count++;
    }
    return true;
}
```

```
bool dequeue(cbuf_t *p_cbuf, int16_t *p_sample)
{
    if (is_empty(p_cbuf))
    {
        return false;
    }

    // Read data from head
    *p_sample = p_cbuf->buffer[p_cbuf->head];

    // advance head with wrap-around
    p_cbuf->head = (p_cbuf->head + 1) % p_cbuf->size;

    // decrement count
    p_cbuf->count--;

    return true;
}

int32_t peek_at(cbuf_t *p_cbuf, int16_t *p_bufout, int32_t p_n_samples,
int32_t p_start_idx)
{
    if (!p_cbuf || !p_bufout || p_n_samples <= 0)
    {
        return 0;
    }

    if (p_start_idx < 0 || p_start_idx >= p_cbuf->size)
    {
        return 0;
    }

    // how many valid samples exist after p_start_idx
    int32_t offset = p_start_idx - p_cbuf->head;
    if (offset < 0)
    {
        offset += p_cbuf->size; // handle wrap-around
    }

    int32_t available_samples;
    if (p_cbuf->count == p_cbuf->size)
    {
        available_samples = p_cbuf->size;
    }
    else
    {
        available_samples = p_cbuf->count - offset;
    }

    // if start_idx is outside the valid written range, nothing to copy
    if (available_samples <= 0)
    {
```

```
        return 0;
    }

    // bound the copy operation to available number of samples
    int32_t samples_to_copy = (p_n_samples > available_samples) ?
available_samples : p_n_samples;
    int32_t samples_to_end = p_cbuf->size - p_start_idx;

    if (samples_to_copy <= samples_to_end)
    {
        // data is contiguous (no wrap-around)
        memcpy(p_bufout, &p_cbuf->buffer[p_start_idx], samples_to_copy *
sizeof(int16_t));
    }
    else
    {
        // data wraps around (need two copies)
        memcpy(p_bufout, &p_cbuf->buffer[p_start_idx], samples_to_end *
sizeof(int16_t));

        int32_t remaining_samples = samples_to_copy - samples_to_end;
        memcpy(p_bufout + samples_to_end, p_cbuf->buffer, remaining_samples *
sizeof(int16_t));
    }

    // return number of elements copied into p_bufout
    return samples_to_copy;
}

typedef float float32_t;

int32_t peek_at_float(cbuf_t *p_cbuf, float32_t *p_bufout, int32_t
p_n_samples, int32_t p_start_idx)
{
    if (!p_cbuf || !p_bufout || p_n_samples <= 0)
    {
        return 0;
    }

    if (p_start_idx < 0 || p_start_idx >= p_cbuf->size)
    {
        return 0;
    }

    // how many valid samples exist after p_start_idx
    int32_t offset = p_start_idx - p_cbuf->head;
    if (offset < 0)
    {
        offset += p_cbuf->size; // handle wrap-around
    }

    int32_t available_samples;
```

```
    if (p_cbuf->count == p_cbuf->size)
    {
        available_samples = p_cbuf->size;
    }
    else
    {
        available_samples = p_cbuf->count - offset;
    }

    if (available_samples <= 0)
    {
        return 0;
    }

    int32_t samples_to_copy = (p_n_samples > available_samples) ?
available_samples : p_n_samples;
    int32_t samples_to_end = p_cbuf->size - p_start_idx;

    if (samples_to_copy <= samples_to_end)
    {
        int32_t i;
        #pragma MUST_ITERATE(1, BUFFER_SIZE, )
        for (i = 0; i < samples_to_copy; i++)
        {
            p_bufout[i] = (float32_t)p_cbuf->buffer[p_start_idx + i];
        }
    }
    else
    {
        int32_t i;
        #pragma MUST_ITERATE(1, BUFFER_SIZE, )
        for (i = 0; i < samples_to_end; i++)
        {
            p_bufout[i] = (float32_t)p_cbuf->buffer[p_start_idx + i];
        }

        int32_t remaining_samples = samples_to_copy - samples_to_end;
        #pragma MUST_ITERATE(1, BUFFER_SIZE, )
        for (i = 0; i < remaining_samples; i++)
        {
            p_bufout[samples_to_end + i] = (float32_t)p_cbuf->buffer[i];
        }
    }

    return samples_to_copy;
}
```

7.2.3 iir.h

```
/**
 * @file iir.h
 * @brief IIR filter structure definitions and functions.
 */
```

```c
#ifndef IIR_H_
#define IIR_H_

#include <stdint.h>

//typedef float float;

#define N_NUM 3
#define N_DEN 2

#define SOS_N 5

/// @brief SOS IIR Filter
/// coeffs stored as:
///
/// b_0, b_1, ..., b_(lnum-1), a1, a2, ..., a(lden-1)
typedef struct iir_filter_t {
    const int nstages;
    const int lnum;
    const int lden;
    const float  g;
    const float* coeffs;
    float tdl[3][SOS_N+1];
    } iir_filter_t;


void sos_filter(
    float input[], uint32_t input_length,
    float output[],
    iir_filter_t* filt
);

void opt_sos_filter(
    float input[], uint32_t input_length,
    float output[],
    iir_filter_t* filt
);

void sos_filter_int(
    int16_t input[], uint32_t input_length,
    int16_t output[],
    iir_filter_t* filt
);
void sos_filter_fast16(
    int16_t input[], uint32_t input_length,
    int16_t output[],
    iir_filter_t* filt
);

#endif
```

7.2.4 iir.c

```c
/**
 * @file iir.c
 * @brief IIR filter structure definitions and functions.
 */
#include "iir.h"

// Tapped delay line for calculating SOS output

// [ x[n],   s0[n],   ..., sN[n] -> y[n] ]
// [ x[n-1], s0[n-1], ..., sN[n-1] -> y[n-1] ]
// [ x[n-2], s0[n-2], ..., sN[n-2] -> y[n-2] ]

// Static so as to persist over calls
//static float tdl[3][SOS_N+1] = {{0,0,0,0,0,0}, {0,0,0,0,0,0},
{0,0,0,0,0,0}};

void sos_filter(
    float input[], uint32_t input_length,
    float output[],
    iir_filter_t* filt
) {
    // TODO: Change this to match the highest of all 3, then use N sos in filt
within.
    //       This will allow filters of different length.
    float num[N_NUM];
    float den[N_DEN];

    unsigned int N = filt->nstages;
    float g = filt->g;
    float (*tdl)[SOS_N+1] = filt->tdl;

    unsigned int xit, sit,cit, tit;
    for (xit = 0; xit < input_length; xit++) {
        tdl[0][0] = input[xit];

        for (sit = 0; sit < N; sit++) {
            // get the filter values
            // Stored as 1D array of numerator1, denominator1, num2, den2, etc
            for (cit = 0; cit < N_NUM; cit++) {
            num[cit] = filt->coeffs[sit*(N_NUM+N_DEN)         + cit];
            };
            for (cit = 0; cit < N_DEN; cit++) {
            den[cit] = filt->coeffs[sit*(N_NUM+N_DEN) + N_NUM + cit];
            };


            // y[n] = b0x[n] + b1x[n-1] + b2x[n-2] -a1y[n-1] - a2y[n-2]
            // In TDL: n is row 0, n-1 row 1, n-2 row 2,
            // x[k] is tdl[][sit], y[k] is tdl[][sit+1]
            tdl[0][sit+1] = num[0]*tdl[0][sit]
```

```
                            + num[1]*tdl[1][sit]
                            + num[2]*tdl[2][sit]
                            - den[0]*tdl[1][sit+1]
                            - den[1]*tdl[2][sit+1];
        }

        // y[n] is output of last stage
        // multiply by gain
        output[xit] = g*tdl[0][SOS_N];

        // TDL update
        // TDL[0][] -> TDL[1][], TDL[1][] -> TDL[2][]
        // set TDL[0][] -> 0
        // We're gonna do this the slow way :))
        for (tit = 0; tit < SOS_N+1;  tit++) {
            tdl[2][tit] = tdl[1][tit];
            tdl[1][tit] = tdl[0][tit];
            tdl[0][tit] = 0.0f;
        };
    };
}

void opt_sos_filter(
    float input[], uint32_t input_length,
    float output[],
    iir_filter_t* filt
) {
    // TODO: Change this to match the highest of all 3, then use N sos in filt
within.
    //       This will allow filters of different length.
    float num[N_NUM];
    float den[N_DEN];

    unsigned int N = filt->nstages;
    float g = filt->g;
    const float *coeffs = filt->coeffs;
    float (*tdl)[SOS_N+1] = filt->tdl;

    unsigned int xit, sit,cit, tit;
    for (xit = 0; xit < input_length; xit++) {
        tdl[0][0] = input[xit];


        for (sit = 0; sit < N; sit++) {
            // Stored as 1D array of numerator1, denominator1, num2, den2, etc
            // opt: Unrolled Loop
            // opt: access coeffs once to avoid the indirection of filt* ->
coeff*
            num[0] = coeffs[sit*(N_NUM+N_DEN)         + 0];
            num[1] = coeffs[sit*(N_NUM+N_DEN)         + 1];
            num[2] = coeffs[sit*(N_NUM+N_DEN)         + 2];
            den[0] = coeffs[sit*(N_NUM+N_DEN) + N_NUM + 0];
```

```
            den[1] = coeffs[sit*(N_NUM+N_DEN) + N_NUM + 1];

            // y[n] = b0x[n] + b1x[n-1] + b2x[n-2] -a1y[n-1] - a2y[n-2]
            // In TDL: n is row 0, n-1 row 1, n-2 row 2,
            // x[k] is tdl[][sit], y[k] is tdl[][sit+1]
            tdl[0][sit+1] = num[0]*tdl[0][sit]
                            + num[1]*tdl[1][sit]
                            + num[2]*tdl[2][sit]
                            - den[0]*tdl[1][sit+1]
                            - den[1]*tdl[2][sit+1];
        }

        // y[n] is output of last stage
        // multiply by gain
        output[xit] = g*tdl[0][SOS_N];

        // TDL update
        // TDL[0][] -> TDL[1][], TDL[1][] -> TDL[2][]
        // set TDL[0][] -> 0
        // We're gonna do this the slow way :))
        for (tit = 0; tit < SOS_N+1;  tit++) {
            tdl[2][tit] = tdl[1][tit];
            tdl[1][tit] = tdl[0][tit];
            tdl[0][tit] = 0.0f;
        };
    };

}
```

7.2.5 state_machine.h

```
/*
 * state_machine.h
 *
 */

#ifndef STATE_MACHINE_H_
#define STATE_MACHINE_H_

#include <stdbool.h>
#include "cbuf.h"
#include "data.h"
#include "iir.h"

#define AUDIOBUF_LEN 500
#define SAMPLE_RATE 8000        // hertz
#define RECORDING_TIME_S 4      // seconds
#define BUFFER_SIZE SAMPLE_RATE * RECORDING_TIME_S

typedef float float32_t;

typedef enum {
```

```
    // S2:1 = OFF
    // Behavior: Standby, no audio output, both LEDs OFF.
    SYS_STATE_STANDBY,

    // S2:1 = ON, S2:2 = OFF
    // Behavior: Record 4s to circular buffer at 8kHz, loopback LINE IN to
LINE OUT.
    // Both LEDs toggle at 20Hz.
    SYS_STATE_RECORD,

    // S2:1 = ON, S2:2 = ON
    // Behavior: Playback 4s buffer circularly. LEDs toggles at 2Hz.
    // Audio output depends on S2:6-8.
    SYS_STATE_PLAYBACK

} system_state_t;


extern volatile system_state_t g_sys_state;
extern volatile bool g_audio_buf_ready;

extern int16_t g_data[BUFFER_SIZE];
extern int16_t g_audio_out[BUFFER_SIZE];

extern cbuf_t cbuf;


#endif /* STATE_MACHINE_H_ */
```

7.2.6 state_machine.c

```
/**
 * @file state_machine.c
 * @brief state machine, software interrupts, and filter application
 */
//-----------------------------------------------------------
// INCLUDES
//-----------------------------------------------------------
#include "maincfg.h" //BIOS include file
#include "framework.h"
#include "state_machine.h"
#include "profiling.h"
#include "data.h"
#include "iir.h"
#include "cbuf.h"
#include <string.h>

#define MAX_ACTIVE_FILTERS 3

//-----------------------------------------------------------
// GLOBAL DECLARATIONS
//-----------------------------------------------------------
```

```c
//-----------------------------------------------------------
// STATIC DECLARATIONS
//-----------------------------------------------------------

static iir_filter_t lp_filt =
{
      lp_N_SOS,
      lp_N_NUM,
      lp_N_DEN,
      lp_G_SOS,
      lp_coeffs,
};


static iir_filter_t hp_filt =
{
      hp_N_SOS,
      lp_N_NUM,
      lp_N_DEN,
      hp_G_SOS,
      hp_coeffs,
};


static iir_filter_t bp_filt =
{
      bp_N_SOS,
      lp_N_NUM,
      lp_N_DEN,
      bp_G_SOS,
      bp_coeffs,
};

#pragma DATA_ALIGN(g_audio_in, 8);
static float32_t g_audio_in[AUDIOBUF_LEN] = {0};
#pragma DATA_ALIGN(g_audio_buf, 8);
static float32_t g_audio_buf[AUDIOBUF_LEN] = {0};

static iir_filter_t *g_active_filters[MAX_ACTIVE_FILTERS] = {0};
static volatile int32_t g_current_read_idx = 0;


//-----------------------------------------------------------
// STATIC FUNCTIONS
//-----------------------------------------------------------

static void apply_parallel_filters(iir_filter_t **p_active_filters, uint8_t
p_num_filters, cbuf_t *p_cbuf, float32_t *p_audio_in, float32_t *p_audio_temp,
int16_t *p_audio_out);
static void apply_parallel_filters_opt(iir_filter_t **p_active_filters,
uint8_t p_num_filters, cbuf_t *p_cbuf, float32_t *p_audio_in, float32_t
```

```
*p_audio_temp, int16_t *p_audio_out);

static void playback_filter_selection(void);
static void profile_filter_permutations(void);


//-----------------------------------------------------------
// SWI ISR
//-----------------------------------------------------------

// toggles at 20 Hz
// handles LED output state
void dipPRD0(void)
{
    if( g_sys_state == SYS_STATE_RECORD)
    {
        LED_toggle(LED_1);
        LED_toggle(LED_2);
    }
}

// toggles at 6 Hz
// handles LED output state
void dipPRD1(void)
{
    if( g_sys_state == SYS_STATE_PLAYBACK)
    {
        LED_toggle(LED_1);
        LED_toggle(LED_2);
    }
}

// executes every 50 ms
// updates system state and audio output buffer
void dipState(void)
{
    uint8_t l_dip_status1, l_dip_status2;
    DIP_get(DIP_1, &l_dip_status1);
    DIP_get(DIP_2, &l_dip_status2);

    // SYS_STATE_STANDBY
    if(!l_dip_status1)
    {
        if( g_sys_state != SYS_STATE_STANDBY)
        {
            LED_turnOff(LED_1);
            LED_turnOff(LED_2);
        }
        g_sys_state = SYS_STATE_STANDBY;
    }
    // SYS_STATE_RECORD
    else if(l_dip_status1 && !l_dip_status2)
```

```
    {
        if( g_sys_state == SYS_STATE_PLAYBACK || g_sys_state ==
SYS_STATE_STANDBY)
        {
            // reset circular buffer
            g_audio_buf_ready = 0;
            init_buffer(&cbuf, g_data);
        }
        g_sys_state = SYS_STATE_RECORD;
    }
    // SYS_STATE_PLAYBACK
    else if(l_dip_status1 && l_dip_status2)
    {
        // wait until buffer has been filled
        if(!is_full(&cbuf))
        {
            g_sys_state = SYS_STATE_RECORD;
            return;
        }
        playback_filter_selection();
        g_sys_state = SYS_STATE_PLAYBACK;
    }
    else
    {
        g_sys_state = SYS_STATE_STANDBY;
    }
    return;
}


static void playback_filter_selection(void)
{
    // CPU cycle profiling
    clock_t start, stop, overhead;
    start = clock();
    stop = clock();
    overhead = stop - start;

    // get filter selection status
    uint8_t l_dip_status6, l_dip_status7, l_dip_status8;
    DIP_get(DIP_6, &l_dip_status6);
    DIP_get(DIP_7, &l_dip_status7);
    DIP_get(DIP_8, &l_dip_status8);

    // array of ptrs to active filters
    iir_filter_t* l_active_filters[MAX_ACTIVE_FILTERS] = { 0 };
    uint8_t l_num_filters = 0;

    // select relevant filters
    if(l_dip_status6)
    {
        l_active_filters[l_num_filters++] = &lp_filt;
```

```
    }
    if(l_dip_status7)
    {
        l_active_filters[l_num_filters++] = &bp_filt;
    }
    if(l_dip_status8)
    {
        l_active_filters[l_num_filters++] = &hp_filt;
    }

    // check if active filters have changed
    if(memcmp(l_active_filters, g_active_filters, sizeof(l_active_filters)) ==
0 && g_audio_buf_ready == 1)
    {
        return; // active filters haven't changed, output buffered audio -->
exit early
    }
    memcpy(g_active_filters, l_active_filters,
MAX_ACTIVE_FILTERS*sizeof(iir_filter_t *));

    // produce new audio output, reset flag
    g_audio_buf_ready = 0;

    // perform parallel IIR filtering
    apply_parallel_filters_opt(l_active_filters, l_num_filters, &cbuf,
g_audio_in, g_audio_buf, g_audio_out);

//    profile_filter_permutations();

    // audio output ready
    g_audio_buf_ready = 1;
    return;
}

static void apply_parallel_filters(iir_filter_t **p_active_filters, uint8_t
p_num_filters, cbuf_t *p_cbuf, float32_t *p_audio_in, float32_t *p_audio_temp,
int16_t *p_audio_out)
{
    int bufit;
    int32_t l_total_samples_read = 0;
    int32_t l_samples_read = 0;
    int32_t i, filter_n;

    // reset audio output buffer
    memset(p_audio_out, 0, BUFFER_SIZE* sizeof(int16_t));

    // return if no active filters
    if(p_num_filters == 0)
    {
        return;
    }
```

```
    // initialise tdl of active filters
    for (filter_n = 0; filter_n < p_num_filters; filter_n++)
    {
        memset(p_active_filters[filter_n]->tdl, 0,
sizeof(p_active_filters[filter_n]->tdl));
    }

    // filter audio buffer in chunks
    for (bufit = 0; bufit < BUFFER_SIZE / AUDIOBUF_LEN; bufit++)
    {
        l_samples_read = peek_at_float(p_cbuf, p_audio_in, AUDIOBUF_LEN,
g_current_read_idx);
        g_current_read_idx = (g_current_read_idx + l_samples_read) % p_cbuf-
>size;

        float32_t l_audio_sum[AUDIOBUF_LEN] = {0};

        // apply all active filters to this chunk
        for (filter_n = 0; filter_n < p_num_filters; filter_n++)
        {
            // apply filter
            sos_filter(p_audio_in, l_samples_read, p_audio_temp,
p_active_filters[filter_n]);
            for (i = 0; i < l_samples_read; i++)
            {
                l_audio_sum[i] += p_audio_temp[i];
            }
        }
        // sum into final audio output
        for (i = 0; i < l_samples_read; i++)
        {
            float32_t l_sample_f = l_audio_sum[i];

            // clip final sum to prevent integer overflow
            if (l_sample_f > 32767.0f) {
                l_sample_f = 32767.0f;
            } else if (l_sample_f < -32768.0f) {
                l_sample_f = -32768.0f;
            }

            p_audio_out[l_total_samples_read + i] += (int16_t)l_sample_f;
        }
        l_total_samples_read += l_samples_read;
    }

}

static void apply_parallel_filters_opt(iir_filter_t **restrict
p_active_filters, uint8_t p_num_filters, cbuf_t *restrict p_cbuf, float32_t
*restrict p_audio_in, float32_t *restrict p_audio_temp, int16_t *restrict
p_audio_out)
{
```

```
    _nassert((int)p_audio_temp % 8 == 0);
    _nassert((int)p_audio_out % 8 == 0);
    int bufit;
    int32_t l_total_samples_read = 0;
    int32_t l_samples_read = 0;
    int32_t i, filter_n;

    // reset audio output buffer
    memset(p_audio_out, 0, BUFFER_SIZE* sizeof(int16_t));

    // return if no active filters
    if(p_num_filters == 0)
    {
        return;
    }

    // initialise tdl of active filters
    #pragma MUST_ITERATE (1, MAX_ACTIVE_FILTERS, );
    for (filter_n = 0; filter_n < p_num_filters; filter_n++)
    {
        memset(p_active_filters[filter_n]->tdl, 0,
sizeof(p_active_filters[filter_n]->tdl));
    }

    // filter audio buffer in chunks
    for (bufit = 0; bufit < BUFFER_SIZE / AUDIOBUF_LEN; bufit++)
    {
        l_samples_read = peek_at_float(p_cbuf, p_audio_in, AUDIOBUF_LEN,
g_current_read_idx);
        g_current_read_idx = (g_current_read_idx + l_samples_read) % p_cbuf-
>size;

        float32_t l_audio_sum[AUDIOBUF_LEN] = {0};

        // apply all active filters to this chunk
        for (filter_n = 0; filter_n < p_num_filters; filter_n++)
        {
            // apply filter
            opt_sos_filter(p_audio_in, l_samples_read, p_audio_temp,
p_active_filters[filter_n]);

            #pragma UNROLL(4)
            #pragma MUST_ITERATE (AUDIOBUF_LEN, AUDIOBUF_LEN,4);
            for (i = 0; i < l_samples_read; i++)
            {
                l_audio_sum[i] += p_audio_temp[i];
            }
        }
        // sum into final audio output
        #pragma UNROLL(2)
        #pragma MUST_ITERATE(2, AUDIOBUF_LEN / 2, 2)
        for (i = 0; i < l_samples_read; i += 2)
```

```
        {
            float32_t f1 = l_audio_sum[i];
            float32_t f2 = l_audio_sum[i+1];

            // cast from float to int32_t
            int32_t i1 = _spint(f1);
            int32_t i2 = _spint(f2);

            // convert to int16_t, saturate, and pack
            int32_t packed_new_int_16 = _spack2(i2, i1);

            // get existing samples
            int32_t packed_existing_int_16 =
_amem4(&p_audio_out[l_total_samples_read + i]);

            // sum samples with saturation
            int32_t packed_sum_16 = _sadd2(packed_existing_int_16,
packed_new_int_16);

            // store
            _amem4(&p_audio_out[l_total_samples_read + i]) = packed_sum_16;
        }
        l_total_samples_read += l_samples_read;
    }

}


static void profile_filter_permutations(void)
{
    clock_t start, stop, overhead;
    start = clock();
    stop = clock();
    overhead = stop - start;
    uint32_t cycles_nonopt, cycles_opt;

    LOG_printf(&trace, "--- Start Filter Profiling ---\r\n");
    uint8_t permutation;
    // Loop from 0 to 7 (Binary 000 to 111) to cover all permutations
    for (permutation = 0; permutation < 8; permutation++)
    {
        // simulate dip switch states
        uint8_t sim_dip6 = (permutation & 0x01) ? 1 : 0; // Bit 0 (Low-Pass)
        uint8_t sim_dip7 = (permutation & 0x02) ? 1 : 0; // Bit 1 (Band-Pass)
        uint8_t sim_dip8 = (permutation & 0x04) ? 1 : 0; // Bit 2 (High-Pass)

        // active filter ptrs
        iir_filter_t* l_active_filters[MAX_ACTIVE_FILTERS] = { 0 };
        uint8_t l_num_filters = 0;

        // select filters based on simulated dip
        if (sim_dip6) l_active_filters[l_num_filters++] = &lp_filt;
```

```
        if (sim_dip7) l_active_filters[l_num_filters++] = &bp_filt;
        if (sim_dip8) l_active_filters[l_num_filters++] = &hp_filt;

        // profile non-optimized function
        start = clock();
        apply_parallel_filters(l_active_filters, l_num_filters, &cbuf,
g_audio_in, g_audio_buf, g_audio_out);
        stop = clock();
        cycles_nonopt = stop - start - overhead;

        // profile optimized function
        start = clock();
        apply_parallel_filters_opt(l_active_filters, l_num_filters, &cbuf,
g_audio_in, g_audio_buf, g_audio_out);
        stop = clock();
        cycles_opt = stop - start - overhead;

        // LOG results
        LOG_printf(&trace, "Permutation -> LP:%d BP:%d", sim_dip6, sim_dip7);
        LOG_printf(&trace, " HP:%d\r\n", sim_dip8);

        LOG_printf(&trace, "  Non-Opt Cycles: %u\r\n", cycles_nonopt);
        LOG_printf(&trace, "  Opt Cycles:     %u\r\n", cycles_opt);
    }

    LOG_printf(&trace, "--- Profiling Complete ---\r\n");
}
```

7.2.7 framework.h

```
//-----------------------------------------------------------------------------
// \file framework.h
// \brief common functions and definitions
//
//-----------------------------------------------------------------------------
#ifndef MAIN_H_
#define MAIN_H_
//-----------------------------------------------------------------------------
// Includes
//-----------------------------------------------------------------------------
#include <std.h>
#include "types.h"
#include "stdio.h"
#include "evmc6748.h"
// #include "evmc6748_timer.h"
#include "evmc6748_gpio.h"
#include "evmc6748_i2c.h"
#include "evmc6748_mcasp.h"
#include "evmc6748_aic3106.h"
#include "evmc6748_led.h"
#include "evmc6748_dip.h"
//-----------------------------------------------------------------------------
// Definitions
```

```
//-----------------------------------------------------------------------------
#define PINMUX_MCASP_REG_17 17
#define PINMUX_MCASP_MASK_17 0x0000ff00
#define PINMUX_MCASP_VAL_17 0x00000800
//#define FS_48KHZ
 #define FS_8KHZ
// #define FS_10KHZ
// #define FS_24KHZ
//-----------------------------------------------------------------------------
//  Prototypes
//-----------------------------------------------------------------------------
void initAll(void);
void isrAudio(void);
void McASP_Init();
void AIC3106_Init();
void McASP_Start();
void USTIMER_delay(uint32_t time);
void SetGpio();
int16_t read_audio_sample();
void write_audio_sample(int16_t);
extern LgUns CLK_gethtime(void);
//-----------------------------------------------------------------------------
// Externs
//-----------------------------------------------------------------------------
extern cregister volatile unsigned int CSR; // control status register
extern cregister volatile unsigned int ICR; // interrupt clear register
extern cregister volatile unsigned int IER; // interrupt enable reg.
extern cregister volatile unsigned int AMR; // Addressing Mode reg
#endif
```

7.2.8 framework.c

```
//-----------------------------------------------------------------------------
// framework.c
// Author: Ariel Jaffe
//-----------------------------------------------------------------------------
//-----------------------------------------------------------------------------
// Includes
//-----------------------------------------------------------------------------
#include "framework.h"
//-----------------------------------------------------------------------------
// DEFINES
//-----------------------------------------------------------------------------
#define RDATA 0x20 // R/XDATA interrupt mask
#define XDATA 0x20
#define I2C_PORT_AIC3106 (I2C0)
//-----------------------------------------------------------------------------
// Private Defines and Macros
//-----------------------------------------------------------------------------
// pinmux defines.
#define PINMUX_MCASP_REG_0 (0)
#define PINMUX_MCASP_MASK_0 (0x00FFFFFF)
#define PINMUX_MCASP_VAL_0 (0x00111111)
```

```
#define PINMUX_MCASP_REG_1 (1)
#define PINMUX_MCASP_MASK_1 (0x000FF000)
#define PINMUX_MCASP_VAL_1 (0x00011000)
//-----------------------------------------------------------------------------
// read_audio_sample()
//-----------------------------------------------------------------------------
int16_t read_audio_sample()
{
    int16_t s;
    s = (int16_t)(MCASP->XBUF12);
    return s;
}
//-----------------------------------------------------------------------------
// write_audio_sample()
//-----------------------------------------------------------------------------
void write_audio_sample(int16_t s)
{
    MCASP->XBUF11 = (uint32_t)s;
}
//-----------------------------------------------------------------------------
// initAll()
//-----------------------------------------------------------------------------
void initAll(void)
{
    I2C_init(I2C0, I2C_CLK_400K); // init I2C channel
    SetGpio();
    LED_init(); // init LED and DIP BSL
    DIP_init();
    McASP_Init();     // init McASP (modified from original BSL)
    AIC3106_Init();   // init AIC3106 (modified from original BSL)
    ICR = (1 << 5);   // clear INT5 (precaution)
    IER |= (1 << 5);  // enable INT5 as CPU interrupt
    IER |= (1 << 11); // enable RTDX interrupts
    IER |= (1 << 12); // enable RTDX interrupts
    McASP_Start();    // start McASP clocks
}
//-----------------------------------------------------------------------------
// McASP_Init()
//-----------------------------------------------------------------------------
void McASP_Init(void)
{
    // enable the psc and config pinmux for mcasp.
    EVMC6748_lpscTransition(PSC1, DOMAIN0, LPSC_MCASP0, PSC_ENABLE);
    EVMC6748_pinmuxConfig(PINMUX_MCASP_REG_0, PINMUX_MCASP_MASK_0,
PINMUX_MCASP_VAL_0);
    EVMC6748_pinmuxConfig(PINMUX_MCASP_REG_1, PINMUX_MCASP_MASK_1,
PINMUX_MCASP_VAL_1);
    // reset mcasp.
    MCASP->GBLCTL = 0;
    // NOTE: ROR 16-bits enabled for both XMT/RCV. SLOT SIZE = 16 bits, 1-bit
delay for Rx
    // clock and frame sync generated by AIC3106. Tx and Rx synchronized
```

```
    // configure receive registers for I2S
    MCASP->RMASK = 0xFFFFFFFF;   // all 32-bits NOT masked
    MCASP->RFMT = 0x0001807C;    // MSB first, align left, slot=16bits, 1-bit
delay, ROR 16-bits
    MCASP->AFSRCTL = 0x00000100; // frame sync generated externally, FS/word,
2 SLOT TDM = I2S
    MCASP->RTDM = 0x00000003;    // SLOT 0 & 1 active I2S
    MCASP->RINTCTL = 0x00000000; // ints disabled
    MCASP->RCLKCHK = 0x00ff0008; // RMAX = FF, RPS = /256
    // configure transmit registers for I2S
    MCASP->XMASK = 0xFFFFFFFF;   // all 32-bits NOT masked
    MCASP->XFMT = 0x0000807C;    // MSB first, align left, slot=16bits, no
delay, ROR 16-bits
    MCASP->AFSXCTL = 0x00000100; // frame sync generated externally, FS/word,
2 SLOT TDM = I2S
    MCASP->XTDM = 0x00000003;    // SLOT 0 & 1 active I2S
    MCASP->XINTCTL = 0x00000000; // ints disabled
    MCASP->XCLKCHK = 0x00ff0008; // RMAX = FF, RPS = /256
    // configure clock operation
    MCASP->ACLKRCTL = 0x00000000;
    MCASP->AHCLKRCTL = 0x00000000;
    MCASP->ACLKXCTL = 0x00000000; // rising edge, clkrm external sync between
Tx and Rx clocks
    MCASP->AHCLKXCTL = 0x00000000;
    // config serializers (11 = xmit, 12 = rcv)
    MCASP->SRCTL11 = 0x000D; // XMT
    MCASP->SRCTL12 = 0x000E; // RCV
    // config pin function and direction.
    MCASP->PFUNC = 0;
    MCASP->PDIR = 0x00000800; // ACLKX and AFSX are input pins
    MCASP->DITCTL = 0x00000000;
    MCASP->DLBCTL = 0x00000000;
    MCASP->AMUTE = 0x00000000;
}
//-----------------------------------------------------------------------------
// McASP_Start_TTO()
//-----------------------------------------------------------------------------
void McASP_Start(void)
{
    // enable the audio clocks, verifying each bit is properly set.
    SETBIT(MCASP->XGBLCTL, XHCLKRST);
    while (!CHKBIT(MCASP->XGBLCTL, XHCLKRST))
    {
    }
    SETBIT(MCASP->RGBLCTL, RHCLKRST);
    while (!CHKBIT(MCASP->RGBLCTL, RHCLKRST))
    {
    }
    SETBIT(MCASP->XGBLCTL, XCLKRST);
    while (!CHKBIT(MCASP->XGBLCTL, XCLKRST))
    {
    }
```

```
    SETBIT(MCASP->RGBLCTL, RCLKRST);
    while (!CHKBIT(MCASP->RGBLCTL, RCLKRST))
    {
    }
    SETBIT(MCASP->RINTCTL, RDATA); // enable McASP XMT/RCV interrupts
    while (!CHKBIT(MCASP->RINTCTL, RDATA))
    {
    } // see #defines at top of file
    /*
    SETBIT(MCASP->XINTCTL, XDATA);
    while (!CHKBIT(MCASP->XINTCTL, XDATA)) {}
    */
    MCASP->XSTAT = 0x0000FFFF; // Clear all (see procedure in UG)
    MCASP->RSTAT = 0x0000FFFF; // Clear all
    SETBIT(MCASP->XGBLCTL, XSRCLR);
    while (!CHKBIT(MCASP->XGBLCTL, XSRCLR))
    {
    }
    SETBIT(MCASP->RGBLCTL, RSRCLR);
    while (!CHKBIT(MCASP->RGBLCTL, RSRCLR))
    {
    }
    /* Write a 0, so that no underrun occurs after releasing the state machine
*/
    MCASP->XBUF11 = 0;
    SETBIT(MCASP->XGBLCTL, XSMRST);
    while (!CHKBIT(MCASP->XGBLCTL, XSMRST))
    {
    }
    SETBIT(MCASP->RGBLCTL, RSMRST);
    while (!CHKBIT(MCASP->RGBLCTL, RSMRST))
    {
    }
    SETBIT(MCASP->XGBLCTL, XFRST);
    while (!CHKBIT(MCASP->XGBLCTL, XFRST))
    {
    }
    SETBIT(MCASP->RGBLCTL, RFRST);
    while (!CHKBIT(MCASP->RGBLCTL, RFRST))
    {
    }
    // wait for transmit ready and send a dummy byte.
    while (!CHKBIT(MCASP->SRCTL11, XRDY))
    {
    }
    MCASP->XBUF11 = 0;
}
//-----------------------------------------------------------------------------
// AIC3106_Init()
//-----------------------------------------------------------------------------
void AIC3106_Init(void)
{
```

```
    // select page 0 and reset codec.
    AIC3106_writeRegister(AIC3106_REG_PAGESELECT, 0);
    AIC3106_writeRegister(AIC3106_REG_RESET, 0x80);
// config codec regs. please see AIC3106 documentation for explanation.
// Document Num: TLV320AIC3106
#ifdef FS_48KHZ // 48kHz sampling rate
    AIC3106_writeRegister(3,
                          (0 << 7) |     // PLL is disabled
                              (4 << 3) | // PLL Q value = 4
                              (2 << 0)); // PLL P value = 2
    // 0x22); // PLL disabled, Q=4, P=2
    AIC3106_writeRegister(2,
                          (0 << 4) |     // ADC fs = fs(ref)/1
                              (0 << 0)); // DAC fs = fs(ref)/1
// 0x00); //ADC/DAC sample rate = f(s)/1 = MCLK/(128*Q)/1
#endif
#ifdef FS_8KHZ
    AIC3106_writeRegister(3,
                          (0 << 7) |     // PLL is disabled
                              (4 << 3) | // PLL Q value = 4
                              (2 << 0)); // PLL P value = 2
    // AIC3106_writeRegister(3, 0x22); // PLL disabled, Q=4, P=2
    AIC3106_writeRegister(2,
                          (10 << 4) |     // ADC fs = fs(ref)/6
                              (10 << 0)); // DAC fs = fs(ref)/6
// AIC3106_writeRegister(2, 0xAA); //ADC/DAC sample rate = f(s)/6 = MCLK/
(128*Q)/6
#endif
#ifdef FS_10KHZ // 9.6kHz
    AIC3106_writeRegister(3,
                          (0 << 7) |      // PLL is disabled
                              (10 << 3) | // PLL Q value = 10
                              (2 << 0));  // PLL P value = 2
    // AIC3106_writeRegister(3, (0 << 7) | (10 << 3) | (2 << 0)); // PLL
disabled, Q=10, P=2
    AIC3106_writeRegister(2,
                          (2 << 4) |     // ADC fs = fs(ref)/6
                              (2 << 0)); // DAC fs = fs(ref)/6
// AIC3106_writeRegister(2, 0x22); //ADC/DAC sample rate = f(s)/2 = MCLK/
(128*Q)/2
#endif
#ifdef FS_24KHZ
    AIC3106_writeRegister(3,
                          (0 << 7) |     // PLL is disabled
                              (4 << 3) | // PLL Q value = 4
                              (2 << 0)); // PLL P value = 2
    // 0x22); // PLL disabled, Q=4, P=2
    AIC3106_writeRegister(2,
                          (2 << 4) |     // ADC fs = fs(ref)/1
                              (2 << 0)); // DAC fs = fs(ref)/1
// 0x00); //ADC/DAC sample rate = f(s)/1 = MCLK/(128*Q)/1
#endif
```

```
    AIC3106_writeRegister(7,
                          (1 << 7) |     // fs(ref) = 48kHz (needed only for
AGC time constants, not used)
                              (0 << 6) | // ADC dual rate mode is disabled
                              (0 << 5) | // DAC dual rate mode is disabled
                              (1 << 3) | // left DAC datapath plays left
channel input data
                              (1 << 1) | // right DAC datapath plays right
channel input data
                              (0 << 0)); // reserved
    // 0x0A);
    AIC3106_writeRegister(8,
                          (1 << 7) |     // BCLK is input (use "1" for output)
                              (1 << 6) | // WCLK is input (use "1" for output)
                              (0 << 5) | // do no place DOUT in high-z when
inactive
                              (0 << 4) | // BCLK & WCLK disabled in master
mode if code powered down
                              (0 << 3) | // reserved
                              (0 << 2) | // disable 3D effect
                              (0 << 0)); // digital mic support disabled
    // 0xC0); // BCLK and WCLK are output
    AIC3106_writeRegister(9,
                          (0 << 6) |     // serial data bus in i2s mode
                              (0 << 4) | // audio word length 16 bits
                              (0 << 3) | // continuous transfer mode
                              (0 << 2) | // don't resync DAC w/ group delay
variation
                              (0 << 1) | // don't resync ADC w/ group delay
variation
                              (0 << 0)); // resync w/o soft muting
// 0x00); // I2S mode, 32-bit data words, continous xfer mode
// AIC3106_writeRegister(10, 0x00); // data word offset
#if 1 // turn input gain off
    // PGA setting, 0 means 0dB gain
    AIC3106_writeRegister(15,
                          (0 << 7) |     // left ADC PGA is not muted
                              (0 << 0)); // left ADC PGA gain setting = 0 dB
    //(0);
    AIC3106_writeRegister(16,
                          (0 << 7) |     // right ADC PGA is not muted
                              (0 << 0)); // right ADC PGA gain setting = 0 dB
//(0);
#else // turn input gain on
    AIC3106_writeRegister(15,
                          (0 << 7) |      // left ADC PGA is not muted
                              (44 << 0)); // left ADC PGA gain setting = 22 dB
    //(0);
    AIC3106_writeRegister(16,
                          (0 << 7) |      // right ADC PGA is not muted
                              (44 << 0)); // right ADC PGA gain setting = 22
dB
```

```
#endif
    AIC3106_writeRegister(19, 0x04); // left ADC is powered up
    AIC3106_writeRegister(22, 0x04); // right ADC is powered up
    AIC3106_writeRegister(27, 0);    // left AGC maximum gain allowed is 0dB
(AGC not used)
    AIC3106_writeRegister(30, 0);    // right AGC maximum gain allowed is 0dB
(AGC not used)
    AIC3106_writeRegister(37,
                          (1 << 7) |     // left DAC powered up
                              (1 << 6) | // right DAC powered up
                              (2 << 4) | // HPLCOM configured as independent
single-ended output (not used here)
                              (0 << 0)); // reserved
    // 0xE0);
    //  set the DAC gain
    AIC3106_writeRegister(43,
                          (0 << 7) |     // left DAC channel is not muted
                              (0 << 0)); // left DAC gain setting = 0dB
    AIC3106_writeRegister(44,
                          (0 << 7) |     // right DAC channel is not muted
                              (0 << 0)); // left DAC gain setting = 0dB
#if 0
// set the DAC gain
AIC3106_writeRegister(43,
(0 << 7) | // left DAC channel is not muted
(0x28 << 0)); // left DAC gain setting = -20dB
AIC3106_writeRegister(44,
(0 << 7) | // right DAC channel is not muted
(0x28 << 0)); // left DAC gain setting = -20dB
#endif
    AIC3106_writeRegister(82,
                          (1 << 7) |     // DAC_L1 is routed to LEFT_LOP/M
                              (0 << 0)); // DAC_L1 to LEFT_LOP/M analog volum
control
    // 0x80);
    AIC3106_writeRegister(86,
                          (0 << 4) |     // LEFT_LOP/M output level control =
0dB
                              (1 << 3) | // LEFT_LOP/M is not muted
                              (0 << 2) | // reserved, read only
                              (0 << 1) | // read only
                              (1 << 0)); // read only (must write 1 for some
reason)
    // 0x09);
    AIC3106_writeRegister(92,
                          (1 << 7) |     // DAC_R1 is routed to RIGHT_LOP/M
                              (0 << 0)); // DAC_R1 to RIGHT_LOP/M analog volum
control
    // 0x80);
    AIC3106_writeRegister(93,
                          (0 << 4) |     // RIGHT_LOP/M output level control =
0dB
```

```
                                (1 << 3) | // RIGHT_LOP/M is not muted
                                (0 << 2) | // reserved, read only
                                (0 << 1) | // read only
                                (1 << 0)); // read only (must write 1 for some
reason)
    // 0x09);
    AIC3106_writeRegister(101,
                            (0 << 6) |     // read only
                                (0 << 5) | // MFP3 pin as GPI disabled
                                (0 << 3) | // read only
                                (0 << 2) | // MFP2 pin as GPO disabled
                                (0 << 1) | // MFP2 drives low when configured as
GPO
                                (1 << 0)); // CODEC_CLKIN uses CLKDIV_OUT
    // 0x01);
    AIC3106_writeRegister(102,
                            (0 << 6) |     // CLKDIV_IN uses MCLK
                                (0 << 4) | // PLLCLK_IN uses MCLK
                                (0 << 0)); // PLL clock divider N = 16
    // 0); // CLKDIV_IN uses MCLK
}
//-----------------------------------------------------------------------------
// /brief Read data from a register on the AIC3106.
//
// /param uint8_t in_reg_addr: The address of the register to be read from.
//
// /param uint8_t * dest_buffer: Pointer to buffer to store retrieved data.
//
// /return uint32_t ERR_NO_ERROR on sucess
//
//-----------------------------------------------------------------------------
uint32_t AIC3106_readRegister(uint8_t in_reg_addr, uint8_t *dest_buffer)
{
    uint32_t rtn;
    // write the register address that we want to read.
    rtn = I2C_write(I2C_PORT_AIC3106, I2C_ADDR_AIC3106, &in_reg_addr, 1,
SKIP_STOP_BIT_AFTER_WRITE);
    if (rtn != ERR_NO_ERROR)
        return (rtn);
    // clock out the register data.
    rtn = I2C_read(I2C_PORT_AIC3106, I2C_ADDR_AIC3106, dest_buffer, 1,
SKIP_BUSY_BIT_CHECK);
    return (rtn);
}
//-----------------------------------------------------------------------------
// /brief Write a register on the AIC3106.
//
// /param uint8_t in_reg_addr: The address of the register to be written to.
//
// /param uint8_t data: Data to be written to the register
//
// /return uint32_t ERR_NO_ERROR on sucess
```

```
//
//-----------------------------------------------------------------------------
uint32_t AIC3106_writeRegister(uint8_t in_reg_addr, uint8_t in_data)
{
    uint32_t rtn;
    uint8_t i2c_data[2];
    i2c_data[0] = in_reg_addr;
    i2c_data[1] = in_data;
    // write the register that we want to read.
    rtn = I2C_write(I2C_PORT_AIC3106, I2C_ADDR_AIC3106, i2c_data, 2,
SET_STOP_BIT_AFTER_WRITE);
    return (rtn);
}
//-----------------------------------------------------------------------------
// USTIMER_delay()
//
// LogicPD BSL fxn - re-written for a few BSL.c files that need it.
// The original USTIMER_init() is not used because it is NOT BIOS compatible
// and took both timers so that BIOS PRDs would not work. This is a
// workaround.
//
// If you need a "delay" in this app, call this routine with the number
// of usec's of delay you need. It is approximate - not exact.
// value for time<300 is perfect for 1us. We padded it some.
//-----------------------------------------------------------------------------
void USTIMER_delay(uint32_t usec)
{
    volatile LgUns i, start, time, current;
    for (i = 0; i < usec; i++)
    {
        start = CLK_gethtime();
        time = 0;
        while (time < 350)
        {
            current = CLK_gethtime();
            time = current - start;
        }
    }
}
//-----------------------------------------------------------------------------
// SetGpio
// config pinmux for gpio
//-----------------------------------------------------------------------------
void SetGpio(void)
{
    EVMC6748_pinmuxConfig(PINMUX_MCASP_REG_17, PINMUX_MCASP_MASK_17,
PINMUX_MCASP_VAL_17);
    GPIO_setDir(GPIO_BANK7, GPIO_PIN7, GPIO_OUTPUT);
}
```

7.2.9 profiling.h

```c
#ifndef PROFILING_H
#define PROFILING_H

#include <stdint.h>
#include <time.h>

typedef float float32_t;

// profiling definitions
#undef CLOCKS_PER_SEC
#define CLOCKS_PER_SEC (300000000)
extern cregister volatile unsigned int TSCL;
extern cregister volatile unsigned int TSCH;


/**
 * @brief Returns current clock cycle.
 *
 * @return clock_t
 *
 */
clock_t clock(void);

#endif // PROFILING_H
```

7.2.10 profiling.c

```c
/**
 * @file profiling.c
 * @brief CPU cycle profiling
 */

#include "profiling.h"
#include <stdio.h>
#include <math.h>
#include <string.h>

clock_t clock(void)
{
    unsigned int low  = TSCL;
    unsigned int high = TSCH;
    if(high) return (clock_t)-1;
    return low;
}
```

7.2.11 main.c

```c
/**
 * main.c
 */
//-----------------------------------------------------------
```

```c
// INCLUDES
//-----------------------------------------------------------
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

#include "maincfg.h" //BIOS include file
#include "framework.h"

#include "cbuf.h"
#include "state_machine.h"

//-----------------------------------------------------------
// GLOBAL DECLARATIONS
//-----------------------------------------------------------

// current system state
volatile system_state_t g_sys_state = SYS_STATE_STANDBY;
// high when active filter selection has been applied
volatile bool g_audio_buf_ready = 0;

// audio input, circular buffer
#pragma DATA_ALIGN(g_data, 8);
int16_t g_data[BUFFER_SIZE] = { 0 };
// audio output, linear buffer
#pragma DATA_ALIGN(g_audio_out, 8);
int16_t g_audio_out[BUFFER_SIZE] = { 0 };

// circular buffer struct
cbuf_t cbuf =
{
    g_data,
    0,
    0,
    0,
    BUFFER_SIZE,
};


//-----------------------------------------------------------
// STATIC DECLARATIONS
//-----------------------------------------------------------

// index of audio output sample
static volatile int32_t g_audio_buf_idx = 0;


//-----------------------------------------------------------
// STATIC FUNCTIONS
//-----------------------------------------------------------
```

```c
//-----------------------------------------------------------
// MAIN
//-----------------------------------------------------------
void main(void)
{
    initAll();
    LOG_printf(&trace, "Start IIR Filter Demo");

    return; // return to BIOS scheduler
}
//-----------------------------------------------------------
// HWI ISR
//-----------------------------------------------------------
void audioHWI(void)
{
    int16_t s16;
    s16 = read_audio_sample(); // can't be placed in switch-case

    // only play through right channel
    if (MCASP->RSLOT)
    {
        // THIS IS THE LEFT CHANNEL!!!
        write_audio_sample(0);
        return;
    }

    switch(g_sys_state)
    {
    // no output
    case SYS_STATE_STANDBY:
        write_audio_sample(0);
        return;

    // buffer and pass-through audio
    case SYS_STATE_RECORD:
        enqueue(&cbuf, s16);       // add sample to buffer
        write_audio_sample(s16);   // pass through audio
        return;

    // output filtered audio
    case SYS_STATE_PLAYBACK:
        if(g_audio_buf_idx >= BUFFER_SIZE) // wrap-around buffer read
        {
            g_audio_buf_idx = 0;
        }
        write_audio_sample(g_audio_out[g_audio_buf_idx++]);
        return;

    // safety case
    default:
        write_audio_sample(0);
        return;
```

```
    }

}
```

8 Bibliography

9 Contributions

Student, Daniel Stebbings:

- Responsible for the filter design, MATLAB code, and C IIR filter code.
- Contribution: 50%

Student, Preben Rasamanickam:

- Responsible for DSP/BIOS, buffering, state machine, and real-time processing.
- Contribution: 50%